

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ОТЧЕТ
по лабораторной работе №8
по дисциплине «Вычислительная математика»
Тема: Решение систем линейных уравнений методом Якоби.

Студент гр. 4342

Кринкин Н.К.

Преподаватель

Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы.

Реализовать метод Якоби, протестировать его на матрицах различного размера (включая матрицы большого размера), изучить условия сходимости итерационных методов и сравнить точность с существующими реализациями методов решения СЛУ.

Теоретический минимум.

Метод Якоби (метод простых итераций) — это итерационный алгоритм решения систем линейных уравнений $Ax=b$. В отличие от прямых методов (таких как метод Гаусса), он не находит точное решение за конечное число шагов, а строит бесконечную последовательность векторов $x^{(0)}, x^{(1)}, \dots$, которая сходится к точному решению.

Исходная система преобразуется к эквивалентному итерационному виду:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij} x_j^{(k)} \right)$$

Достаточное условие сходимости: Метод Якоби гарантированно сходится при любом начальном приближении $x^{(0)}$, если матрица A обладает **строгим диагональным преобладанием** (модуль диагонального элемента строго больше суммы модулей остальных элементов в строке).

Вычисления прекращаются, когда максимальная разница между текущим и предыдущим приближением становится меньше заданной точности: $\max |x_i^{(k+1)} - x_i^{(k)}| \leq \varepsilon$.

Для оценки итоговой погрешности используется Евклидова норма (L2-норма) вектора ошибки: $e = \|x - x^*\|_2$.

Временная сложность и эффективность метода:

Вычислительная (временная) сложность метода Якоби составляет $O(K \cdot N^2)$, где N — размерность матрицы, а K — количество итераций, необходимых для достижения точности ε .

Обоснование сложности: На каждой отдельной итерации для вычисления одного нового значения x_i требуется выполнить сумму из $N-1$ произведений. Так как вектор состоит из N переменных, один полный шаг обновления всего вектора x требует выполнения ровно $N \cdot (N-1) \approx N^2$ арифметических операций. Для получения итогового ответа этот шаг повторяется K раз.

Преимущества перед прямыми методами: Временная сложность метода Гаусса составляет $O(N^3)$. Если матрица имеет сильное диагональное преобладание, алгоритм Якоби сходится очень быстро, и число итераций K оказывается многократно меньше размера матрицы ($K \ll N$). В таком случае итерационный алгоритм работает на порядки быстрее метода Гаусса. Кроме того, метод Якоби не изменяет исходную матрицу A в процессе вычислений (отсутствует «проблема заполнения» нулей ненулевыми элементами), что делает его идеальным выбором для решения сверхбольших разреженных и трехдиагональных матриц.

Выполнение работы.

В качестве метода решения был реализован метод Якоби. При запуске программы создается квадратная матрица A размера $N \times N$, заполненная случайными числами. Так как метод Якоби расходится для случайных матриц, к сгенерированной матрице принудительно **применяется преобразование для обеспечения диагонального преобладания** (к каждому диагональному элементу прибавляется сумма модулей остальных элементов строки). Затем генерируется эталонный вектор решений x^* , вычисляется вектор свободных членов b , и данные передаются в функцию `Jacobi`. Алгоритм итеративно вычисляет приближение до тех пор, пока разница между шагами не станет меньше заданного ε . Предусмотрена защита от закливания: если число итераций превысит 10000, алгоритм прерывается с ошибкой расходимости.

Тестирование алгоритма:

1. Анализ скорости и точности:

Алгоритм был запущен на матрицах различного значения, а также на матрицах различной точности. График зависимости числа итераций от точности, а также количество итераций от размера матрицы представлены на рисунках 1 и 2.

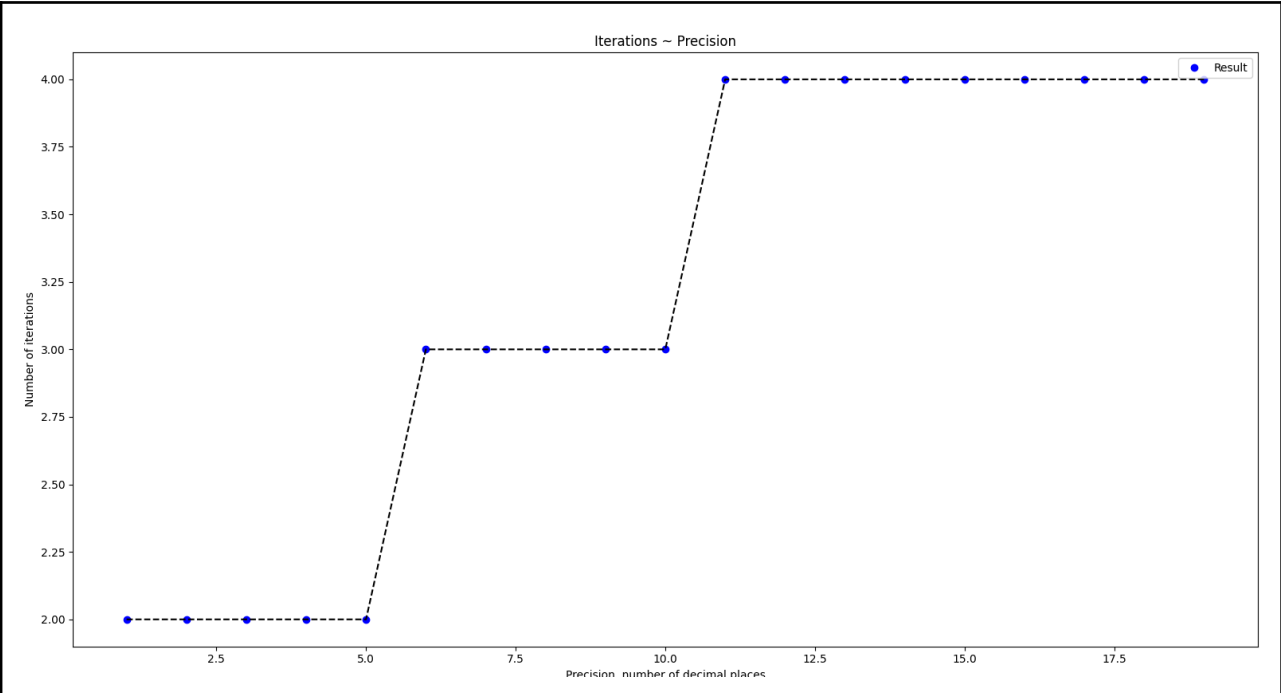


Рисунок 1 – зависимости числа итераций от точности

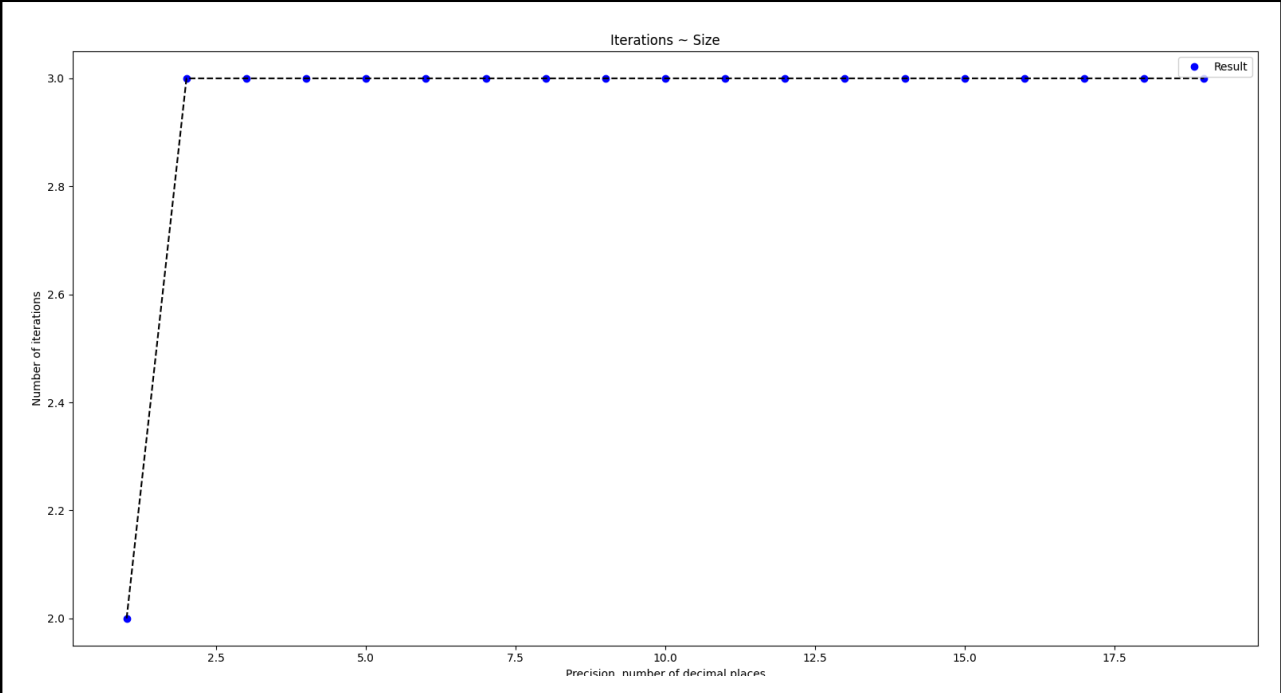


Рисунок 2 – зависимости числа итераций от размера матрицы

Выводы.

Был успешно реализован и протестирован итерационный метод Якоби. Подтверждено теоретическое условие сходимости: алгоритм быстро и с высочайшей точностью находит решение для матриц со строгим диагональным преобладанием, справляясь даже с огромными размерностями (10000×10000). В отличие от универсального метода Гаусса, метод Якоби требует предварительного анализа свойств системы перед применением.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Файл lab.cpp:

```
#include <iostream>
#include <cmath>
#include <random>

#define MAXIMUM_ELEMENT_VALUE 10

int Jacobi(int N, double** A, double* F, double* X, double eps) {
    double* TempX = new double[N]();
    double* X_prev = new double[N];
    for (int i = 0; i < N; i++) {
        X[i] = 0;
    }
    double diff;
    int iterations = 0;
    const long MAX_ITER = 10000;
    do {
        for (int i = 0; i < N; i++) {
            X_prev[i] = X[i];
        }
        for (int i = 0; i < N; i++) {
            double sum = F[i];
            for (int j = 0; j < N; j++) {
                if (i != j) {
                    sum -= A[i][j] * X_prev[j];
                }
            }
            TempX[i] = sum / A[i][i];
        }
        for (int i = 0; i < N; i++) {
            diff = fabs(TempX[i] - X_prev[i]);
        }
        for (int i = 0; i < N; i++) {
            X[i] = TempX[i];
        }
        iterations++;
        if (iterations >= MAX_ITER) {
            std::cerr << "Error: iteration limit!\n";
            break;
        }
    } while (diff > eps);
    delete[] TempX;
    delete[] X_prev;

    return iterations;
}

double getRandomNumber() {
    static std::random_device rd;
    static std::mt19937 gen(rd());
    static std::uniform_real_distribution<double> dist(
        -MAXIMUM_ELEMENT_VALUE, MAXIMUM_ELEMENT_VALUE
    );
    return dist(gen);
}
```

```

double** createMatrix(int N) {
    double** A = new double*[N];
    for (int i = 0; i < N; ++i) {
        A[i] = new double[N];
        double sum = 0;
        for (int j = 0; j < N; ++j) {
            if (i != j + 100) {
                A[i][j] = getRandomNumber();
                sum += fabs(A[i][j]);
            }
        }
        // A[i][i] = sum + 100000000;
    }
    return A;
}

std::pair<double*, double*> getCorrectResponseVectors(double** A, int N) {
    double* X = new double[N];
    double* B = new double[N];
    for (int i = 0; i < N; ++i) {
        X[i] = getRandomNumber();
    }
    for (int i = 0; i < N; ++i) {
        B[i] = 0;
        for (int j = 0; j < N; ++j) {
            B[i] += A[i][j] * X[j];
        }
    }
    return {X, B};
}

void deleteMatrix(double** A, int N) {
    for (int i = 0; i < N; ++i) {
        delete[] A[i];
    }
    delete[] A;
}

int main() {
    int N;
    int pr;
    std::cout << "Enter table size: ";
    std::cin >> N;
    std::cout << "Enter precision (decimal places): ";
    std::cin >> pr;
    auto A = createMatrix(N);
    auto vectors = getCorrectResponseVectors(A, N);
    double* JacobiAnswer = new double[N]();
    int iter = Jacobi(N, A, vectors.second, JacobiAnswer, std::pow(10, -pr));
    std::cout << "Solution found! Iterations -- " << iter << std::endl;
    for (int i = 0; i < N; i++) {
        std::cout << "X[" << i << "] = " << JacobiAnswer[i]
            << " (correct " << vectors.first[i] << ")\n";
    }
    delete[] vectors.first;
    delete[] vectors.second;
    delete[] JacobiAnswer;
    deleteMatrix(A, N);
    return EXIT_SUCCESS;
}

```

Файл research.cpp:

```
#include <iostream>
#include <cmath>
#include <cstdio>
#include <random>

#define MAXIMUM_ELEMENT_VALUE 10

int Jacobi(int N, double** A, double* F, double* X, double eps) {
    double* TempX = new double[N]();
    double* X_prev = new double[N];
    for (int i = 0; i < N; i++) {
        X[i] = 0;
    }
    double diff;
    int iterations = 0;
    const long MAX_ITER = 10000;
    do {
        for (int i = 0; i < N; i++) {
            X_prev[i] = X[i];
        }
        for (int i = 0; i < N; i++) {
            double sum = F[i];
            for (int j = 0; j < N; j++) {
                if (i != j) {
                    sum -= A[i][j] * X_prev[j];
                }
            }
            TempX[i] = sum / A[i][i];
        }
        for (int i = 0; i < N; i++) {
            diff = fabs(TempX[i] - X_prev[i]);
        }
        for (int i = 0; i < N; i++) {
            X[i] = TempX[i];
        }
        iterations++;
        if (iterations >= MAX_ITER) {
            std::cerr << "Error: iteration limit!\n";
            break;
        }
    } while (diff > eps);
    delete[] TempX;
    delete[] X_prev;

    return iterations;
}

double getRandomNumber() {
    static std::random_device rd;
    static std::mt19937 gen(rd());
    static std::uniform_real_distribution<double> dist(
        -MAXIMUM_ELEMENT_VALUE, MAXIMUM_ELEMENT_VALUE
    );
    return dist(gen);
}

double** createMatrix(int N) {
```

```

double** A = new double*[N];
for (int i = 0; i < N; ++i) {
    A[i] = new double[N];
    double sum = 0;
    for (int j = 0; j < N; ++j) {
        if (i != j) {
            A[i][j] = getRandomNumber();
            sum += fabs(A[i][j]);
        }
    }
    A[i][i] = sum + 100000000;
}
return A;
}

std::pair<double*, double*> getCorrectResponseVectors(double** A, int N) {
    double* X = new double[N];
    double* B = new double[N];
    for (int i = 0; i < N; ++i) {
        X[i] = getRandomNumber();
    }
    for (int i = 0; i < N; ++i) {
        B[i] = 0;
        for (int j = 0; j < N; ++j) {
            B[i] += A[i][j] * X[j];
        }
    }
    return {X, B};
}

void deleteMatrix(double** A, int N) {
    for (int i = 0; i < N; ++i) {
        delete[] A[i];
    }
    delete[] A;
}

int main(int argc, char *argv[]) {
    // iteration over precision
    FILE *prec = fopen(argv[1], "w");
    for (int i=1; i<20; i++) {
        auto A = createMatrix(25);
        auto vectors = getCorrectResponseVectors(A, 25);
        double* JacobiAnswer = new double[25]();
        int iter = Jacobi(25, A, vectors.second, JacobiAnswer, std::pow(10, -i));
        fprintf(prec, "%d %d\n", i, iter);
        delete[] vectors.first;
        delete[] vectors.second;
        delete[] JacobiAnswer;
        deleteMatrix(A, 25);
    }
    // iterations over size
    FILE *comp = fopen(argv[2], "w");
    for (int n=1; n<=1000; n++) {
        auto A = createMatrix(n);
        auto vectors = getCorrectResponseVectors(A, n);
        double* JacobiAnswer = new double[n]();
        int iter = Jacobi(n, A, vectors.second, JacobiAnswer, std::pow(10, -8));
        fprintf(comp, "%d %d\n", n, iter);
        delete[] vectors.first;
    }
}

```

```

        delete[] vectors.second;
        delete[] JacobiAnswer;
        deleteMatrix(A, n);
    }
    return EXIT_SUCCESS;
}

```

Файл graph.py:

```

import matplotlib
matplotlib.use("TkAgg")
from matplotlib import pyplot as plt
import sys

pres = open(sys.argv[1], "r").readlines()
comp = open(sys.argv[2], "r").readlines()

xs_p = []
xs_c = []
ys_p = []
ys_c = []

for line1, line2 in zip(pres, comp):
    if line1:
        xp, yp = list(map(int, line1.split()))
        xs_p.append(xp)
        ys_p.append(yp)
    if line2:
        xc, yc = list(map(int, line2.split()))
        xs_c.append(xc)
        ys_c.append(yc)

plt.plot(xs_p, ys_p, "bo", label="Result")
plt.plot(xs_p, ys_p, "k--")
plt.title("Iterations ~ Precision")
plt.xlabel("Precision, number of decimal places")
plt.ylabel("Number of iterations")
plt.legend()
plt.show()

plt.plot(xs_c, ys_c, "bo", label="Result")
plt.plot(xs_c, ys_c, "k--")
plt.title("Iterations ~ Size")
plt.xlabel("Precision, number of decimal places")
plt.ylabel("Number of iterations")
plt.legend()
plt.show()

```

Файл Makefile:

```

SRC_DIR = ${CURDIR}/src
OBJ_DIR = ${CURDIR}/objs
EXP_DIR = ${CURDIR}/examples

FN_PRECS = ${OBJ_DIR}/precs.txt
FN_COMP = ${OBJ_DIR}/comp.txt

lab: ${OBJ_DIR}/lab.o
    ${OBJ_DIR}/lab.o

research: ${SRC_DIR}/graph.py ${FN_PRECS}
    python3 ${SRC_DIR}/graph.py ${FN_PRECS} ${FN_COMP}

```

```
{FN_PRECS}: ${OBJ_DIR}/research.o
    ${OBJ_DIR}/research.o ${FN_PRECS} ${FN_COMP}

${OBJ_DIR}:
    mkdir ${OBJ_DIR}

${OBJ_DIR}/lab.o: ${OBJ_DIR} ${SRC_DIR}/lab.cpp
    @g++ -g3 ${SRC_DIR}/lab.cpp -o ${OBJ_DIR}/lab.o

${OBJ_DIR}/research.o: ${OBJ_DIR} ${SRC_DIR}/research.cpp
    @g++ -g3 ${SRC_DIR}/research.cpp -o ${OBJ_DIR}/research.o

clear:
    rm ${OBJ_DIR}/*
```